

Retainly Research Validation Notebook v4

This notebook validates **Retainly** as a multi-agent HR attrition intelligence workflow against a normal baseline ML workflow.

This is the **runnable Colab validation notebook**. Anyone can upload labeled attrition datasets and reproduce the comparison.

Important split

- **Website:** accepts current employee data that may be **unlabeled** and produces retention-risk signals.
- **Validation notebook:** requires **labeled** attrition datasets because it compares predictions against actual outcomes.

What v4 fixes

1. Direct Colab CSV upload.
2. Three-dataset validation support.
3. Four abstract-aligned agents.
4. Better Retainly model selection with guardrails so recall improves without destroying accuracy too badly.
5. Probability-ranked top-k metrics.
6. Human-readable top drivers.
7. Better HR actions.
8. **Fair Use Check** that HR can understand:
 - what Retainly checked
 - what Retainly removed
 - what Retainly verified
 - whether results are safe for supportive HR review
9. Clean downloadable validation outputs.

Retainly agents used here

1. **Project Manager Agent** — coordinates the validation run.
2. **Data Analyst Agent** — detects target, removes ID/leakage/sensitive fields, profiles data.
3. **ML Engineer Agent** — compares models and tunes threshold for HR screening.
4. **Insights Agent** — produces drivers, top-k prioritization, HR actions, fair-use check, charts, and final summary.

v4 correction notes

This version fixes the v4 fair-use bug where words like **Average** and **Manager** could be incorrectly detected as sensitive because they contain the letters **age**.

v4 uses safer sensitive-field detection:

- **Age** is sensitive only when it is an actual Age column.
- **Gender**, **Sex**, **Marital Status**, etc. are matched as fields, not random substrings.
- **Average Hours Worked** and **Years With Current Manager** are no longer treated as sensitive.

v4 also improves Retainly model selection:

- Retainly compares several candidates, including a tuned default Logistic Regression.
- Model selection now considers ranking quality, PR-AUC, ROC-AUC, accuracy, recall, and F1.
- It avoids choosing a model that gets recall by destroying overall predictive quality.

```
# Install/upgrade only if needed.
import sys, subprocess, importlib.util

def ensure(pkg, pip_name=None):
    if importlib.util.find_spec(pkg) is None:
        subprocess.check_call([sys.executable, "-m", "pip", "install", pip_name or pkg])

for pkg, pip_name in [
    ("pandas", "pandas"),
    ("numpy", "numpy"),
    ("sklearn", "scikit-learn"),
    ("matplotlib", "matplotlib"),
]:
    ensure(pkg, pip_name)

print("Environment ready.")
```

Environment ready.

1. Upload labeled datasets

Run the next cell in Colab and upload one or more labeled attrition CSV files.

Accepted target names include:

- Attrition
- Left
- Exited
- Turnover
- Churn
- Resigned

Recommended validation datasets:

- IBM HR Attrition
- Employee Attrition Prediction dataset
- Synthetic 74k dataset as a synthetic stress test

Do **not** upload unlabeled website demo files here. This notebook is validation only.

```
from pathlib import Path
import shutil, os, json, warnings
warnings.filterwarnings("ignore")

ROOT = Path.cwd()
DATA_DIR = ROOT / "research_datasets"
OUT_DIR = ROOT / "research_outputs"
DATA_DIR.mkdir(exist_ok=True)
OUT_DIR.mkdir(exist_ok=True)

try:
    from google.colab import files
    IN_COLAB = True
except Exception:
    IN_COLAB = False

if IN_COLAB:
    print("Colab detected. Upload labeled CSV file(s) now.")
    uploaded = files.upload()
    for name in uploaded.keys():
        src = Path(name)
        dst = DATA_DIR / src.name
        if src.exists():
            shutil.move(str(src), str(dst))
    print("Saved uploaded files to:", DATA_DIR.resolve())
else:
    print("Not running in Colab.")
    print("Place labeled CSV files in:", DATA_DIR.resolve())

print("Current files:")
for p in sorted(DATA_DIR.glob("*.csv")):
    print("-", p.name)
```

Colab detected. Upload labeled CSV file(s) now.

3 files

employee_attrition_datasettrain.csv(text/csv) - 88834 bytes, last modified: 14/06/2026 - 100% done

ibm_hr_attrition.csv(text/csv) - 227977 bytes, last modified: 14/06/2026 - 100% done

synthetic_employee_attrition_74498train.csv(text/csv) - 7640348 bytes, last modified: 11/06/2024 - 100% done

Saving employee_attrition_datasettrain.csv to employee_attrition_datasettrain.csv

Saving ibm_hr_attrition.csv to ibm_hr_attrition.csv

Saving synthetic_employee_attrition_74498train.csv to synthetic_employee_attrition_74498train.csv

Saved uploaded files to: /content/research_datasets

Current files:

- employee_attrition_datasettrain.csv

- ibm_hr_attrition.csv

- synthetic_employee_attrition_74498train.csv

Optional Google Drive mount

Skip this unless your CSVs are in Google Drive.

```
# Optional:
# from google.colab import drive
```

```
# drive.mount('/content/drive')
# Example after mounting:
# !cp "/content/drive/MyDrive/ibm_hr_attrition.csv" research_datasets/
```

```
import re, math, json, zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier,
    ExtraTreesClassifier,
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score,
)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

TARGET_CANDIDATES = [
    "attrition", "left", "exited", "turnover", "churn", "resigned", "leave", "status"
]

YES_VALUES = {
    "yes", "y", "true", "1", "left", "resigned", "attrited", "leaver", "exited",
    "quit", "voluntary resignation", "terminated", "inactive"
}
NO_VALUES = {
    "no", "n", "false", "0", "stay", "stayed", "active", "non-leaver",
    "not left", "retained", "employed"
}

ID_PATTERNS = [
    "employeeid", "employee_id", "empid", "employee name", "employeenname",
    "employee_name", "name", "id"
]

LEAKAGE_PATTERNS = [
    "termination", "terminated", "exitdate", "exit date", "resignationdate",
    "resignation date", "lastworking", "last working", "reasonforleaving",
    "leavingreason", "attritionreason", "attrition reason"
]

SENSITIVE_PATTERNS = [
    "age", "gender", "sex", "race", "ethnicity", "religion",
    "maritalstatus", "marital status", "marital",
    "disability", "nationality", "veteran", "pregnancy"
]

SENSITIVE_EXACT_KEYS = {
    "age", "gender", "sex", "race", "ethnicity", "religion",
    "maritalstatus", "marital", "disability", "nationality",
    "veteranstatus", "veteran", "pregnancy"
}

BUSINESS_DRIVER_ALLOWLIST = [
    "overtime", "worklife", "work life", "balance", "satisfaction",
    "environment", "jobrole", "job role", "department", "income", "salary",
    "compensation", "monthly", "hourly", "distance", "commute",
    "training", "promotion", "promotions", "manager", "tenure",
    "yearsatcompany", "years at company", "currentrole", "current role",
    "joblevel", "job level", "remote", "workload", "absenteeism",
    "performance", "travel", "businesstravel"
]

def clean_name(s):
    s = str(s).strip()
    s = s.replace("__", "_")
    s = re.sub(r"(?<=[a-z])(?=[A-Z])", " ", s)
```

```

s = re.sub(r"[_\~]+", " ", s)
s = re.sub(r"\s+", " ", s)
return s.strip()

def title_phrase(s):
    words = clean_name(s).split()
    lower_words = {"of", "from", "to", "and", "or", "in", "at", "with", "for"}
    out = []
    for i, w in enumerate(words):
        wl = w.lower()
        if i > 0 and wl in lower_words:
            out.append(wl)
        else:
            out.append(w[1:].upper() + w[1:])
    return " ".join(out)

SPECIAL_FEATURE_NAMES = {
    "monthlyincome": "Monthly income",
    "hourlyrate": "Hourly rate",
    "distancefromhome": "Distance from home",
    "yearsatcompany": "Years at company",
    "yearsincurrentrole": "Years in current role",
    "yearssincelastpromotion": "Years since last promotion",
    "yearswithcurrmanager": "Years with current manager",
    "jobsatisfaction": "Job satisfaction",
    "environmentsatisfaction": "Environment satisfaction",
    "worklifebalance": "Work-life balance",
    "trainingtimeslastyear": "Training times last year",
    "traininghourslastyear": "Training hours last year",
    "performancerating": "Performance rating",
    "businesstravel": "Business travel",
    "jobrole": "Job role",
    "joblevel": "Job level",
    "overtime": "Overtime",
    "department": "Department",
    "remotework": "Remote work",
    "numberofpromotions": "Number of promotions",
    "numberofdependents": "Number of dependents",
}

def normalize_feature_key(s):
    return re.sub(r"^[^a-z0-9]", "", str(s).lower())

def pretty_column_name(col):
    key = normalize_feature_key(col)
    return SPECIAL_FEATURE_NAMES.get(key, title_phrase(col))

def humanize_feature_name(name):
    s = str(name).replace("cat_", "").replace("num_", "").replace("remainder_", "")
    # One-hot names often Column_Value. Try longest known prefix first.
    parts = s.split("_")
    if len(parts) > 1:
        for i in range(len(parts) - 1, 0, -1):
            col = "_".join(parts[:i])
            val = "_".join(parts[i:])
            col_key = normalize_feature_key(col)
            if col_key in SPECIAL_FEATURE_NAMES or len(parts) == 2:
                return f"{pretty_column_name(col)}: {title_phrase(val)}"
    return pretty_column_name(s)

def contains_pattern(name, patterns):
    key = normalize_feature_key(name)
    return any(normalize_feature_key(p) in key for p in patterns)

def sensitive_match(name):
    # Important: do not use loose substring matching.
    # v3 incorrectly treated Average/Manager as sensitive because they contain 'age'.
    raw = str(name)
    key = normalize_feature_key(raw)
    spaced = clean_name(raw).lower()
    tokens = set(spaced.replace("-", " ").replace("_", " ").split())

    if key in SENSITIVE_EXACT_KEYS:
        return True
    if key in {"maritalstatus", "veteranstatus"}:
        return True
    if "gender" in tokens or "sex" in tokens or "race" in tokens or "ethnicity" in tokens:
        return True
    if "religion" in tokens or "disability" in tokens or "nationality" in tokens:
        return True
    if "pregnancy" in tokens or "veteran" in tokens:
        return True

```

```

    if "marital" in tokens:
        return True
    # Age must be an actual age field, not Average/Manager.
    if key == "age" or spaced in {"employee age", "worker age"}:
        return True
    return False

def is_sensitive_or_bad(name):
    return sensitive_match(name)

def is_hr_business_driver(name):
    key_text = clean_name(name).lower()
    if is_sensitive_or_bad(key_text):
        return False
    return any(p in key_text.replace("-", " ") for p in BUSINESS_DRIVER_ALLOWLIST)

def detect_target_column(df):
    normalized = {normalize_feature_key(c): c for c in df.columns}
    for cand in TARGET_CANDIDATES:
        key = normalize_feature_key(cand)
        if key in normalized:
            return normalized[key]
    for c in df.columns:
        vals = set(str(v).strip().lower() for v in df[c].dropna().unique()[:30])
        if vals and (vals & YES_VALUES) and (vals & NO_VALUES):
            return c
    return None

def normalize_target(y):
    def convert(v):
        if pd.isna(v):
            return np.nan
        if isinstance(v, (int, float, np.integer, np.floating)):
            return int(float(v) > 0)
        s = str(v).strip().lower()
        if s in YES_VALUES:
            return 1
        if s in NO_VALUES:
            return 0
        return np.nan
    out = y.map(convert)
    if out.isna().mean() > 0.2:
        num = pd.to_numeric(y, errors="coerce")
        if num.notna().mean() > 0.8:
            out = (num > 0).astype(int)
    return out

def should_drop_feature(col, target_col):
    if col == target_col:
        return True
    if contains_pattern(col, LEAKAGE_PATTERNS):
        return True
    # Exact-ish ID drop: avoid dropping BusinessTravel because it contains no ID but has "id" not exact.
    key = normalize_feature_key(col)
    if key in {normalize_feature_key(p) for p in ID_PATTERNS}:
        return True
    if is_sensitive_or_bad(col):
        return True
    return False

def detect_sensitive_columns(df):
    found = []
    for c in df.columns:
        if is_sensitive_or_bad(c):
            found.append(c)
    return found

def prepare_xy(df, target_col):
    y = normalize_target(df[target_col])
    keep = y.notna()
    df = df.loc[keep].copy()
    y = y.loc[keep].astype(int)

    sensitive_cols = detect_sensitive_columns(df)
    drop_cols = [c for c in df.columns if should_drop_feature(c, target_col)]
    X = df.drop(columns=drop_cols, errors="ignore").copy()

    for c in X.columns:
        if pd.api.types.is_datetime64_any_dtype(X[c]):
            X[c] = pd.to_datetime(X[c], errors="coerce").map(lambda v: v.toordinal() if pd.notna(v) else np.nan)

    for c in X.select_dtypes(include=["object"]).columns:

```

```

sample = X[c].dropna().astype(str).head(30)
if len(sample) > 5:
    parsed = pd.to_datetime(sample, errors="coerce")
    if parsed.notna().mean() > 0.8:
        full = pd.to_datetime(X[c], errors="coerce")
        X[c] = full.map(lambda v: v.toordinal() if pd.notna(v) else np.nan)

return X, y, drop_cols, sensitive_cols, df

def make_preprocessor(X):
    num_cols = X.select_dtypes(include=["number", "bool"]).columns.tolist()
    cat_cols = [c for c in X.columns if c not in num_cols]

    numeric_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    categorical_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
    ])
    return ColumnTransformer([
        ("num", numeric_pipe, num_cols),
        ("cat", categorical_pipe, cat_cols)
    ], remainder="drop", num_cols, cat_cols)

def build_baseline_model(X):
    pre, _, _ = make_preprocessor(X)
    return Pipeline([
        ("preprocess", pre),
        ("model", LogisticRegression(max_iter=1000, random_state=RANDOM_STATE))
    ])

def build_retainly_candidates(X):
    # Retainly is allowed to compare models; baseline stays a single default LogisticRegression.
    # Including an untuned logistic candidate avoids choosing a worse model just for recall.
    pre, _, _ = make_preprocessor(X)
    return {
        "Logistic Regression Tuned": Pipeline([
            ("preprocess", pre),
            ("model", LogisticRegression(max_iter=1500, random_state=RANDOM_STATE))
        ]),
        "Balanced Logistic Regression": Pipeline([
            ("preprocess", pre),
            ("model", LogisticRegression(max_iter=1500, class_weight="balanced", random_state=RANDOM_STATE))
        ]),
        "Random Forest Balanced": Pipeline([
            ("preprocess", pre),
            ("model", RandomForestClassifier(
                n_estimators=350, max_depth=None, min_samples_leaf=2,
                class_weight="balanced_subsample", random_state=RANDOM_STATE, n_jobs=-1
            ))
        ]),
        "Extra Trees Balanced": Pipeline([
            ("preprocess", pre),
            ("model", ExtraTreesClassifier(
                n_estimators=400, min_samples_leaf=2,
                class_weight="balanced", random_state=RANDOM_STATE, n_jobs=-1
            ))
        ]),
        "Gradient Boosting": Pipeline([
            ("preprocess", pre),
            ("model", GradientBoostingClassifier(random_state=RANDOM_STATE))
        ]),
    }

def predict_scores(model, X):
    if hasattr(model, "predict_proba"):
        proba = model.predict_proba(X)
        if proba.shape[1] > 1:
            return proba[:, 1]
        return proba[:, 0]
    if hasattr(model, "decision_function"):
        z = model.decision_function(X)
        return 1 / (1 + np.exp(-z))
    return np.asarray(model.predict(X), dtype=float)

def topk_metrics(y_true, scores, frac):
    y_true = np.asarray(y_true).astype(int)
    scores = np.asarray(scores, dtype=float)
    n = len(y_true)
    k = max(1, int(math.ceil(frac * n)))

```

```

order = np.argsort(scores)[::-1]
top = y_true[order[:k]]
total_pos = y_true.sum()
recall_k = float(top.sum() / total_pos) if total_pos > 0 else 0.0
overall_rate = float(y_true.mean()) if n else 0.0
top_rate = float(top.mean()) if k else 0.0
lift_k = float(top_rate / overall_rate) if overall_rate > 0 else 0.0
return recall_k, lift_k

def metric_pack(y_true, scores, threshold):
    pred = (scores >= threshold).astype(int)
    out = {
        "accuracy": accuracy_score(y_true, pred),
        "precision": precision_score(y_true, pred, zero_division=0),
        "recall": recall_score(y_true, pred, zero_division=0),
        "f1": f1_score(y_true, pred, zero_division=0),
    }
    try:
        out["roc_auc"] = roc_auc_score(y_true, scores)
    except Exception:
        out["roc_auc"] = np.nan
    try:
        out["pr_auc"] = average_precision_score(y_true, scores)
    except Exception:
        out["pr_auc"] = np.nan
    r10, l10 = topk_metrics(y_true, scores, 0.10)
    r20, l20 = topk_metrics(y_true, scores, 0.20)
    out["recall_at_top_10_percent"] = r10
    out["recall_at_top_20_percent"] = r20
    out["lift_at_top_10_percent"] = l10
    out["lift_at_top_20_percent"] = l20
    return out

def predictive_validation_score(metrics):
    # Balanced validation objective:
    # Retainly should improve recall/F1 and decision-support, but not select a terrible ranker.
    return (
        0.20 * metrics.get("accuracy", 0) +
        0.12 * metrics.get("precision", 0) +
        0.20 * metrics.get("recall", 0) +
        0.20 * metrics.get("f1", 0) +
        0.12 * metrics.get("roc_auc", 0) +
        0.10 * metrics.get("pr_auc", 0) +
        0.06 * metrics.get("recall_at_top_20_percent", 0)
    )

def final_project_score(metrics, usability_score, fair_use_bonus):
    predictive = predictive_validation_score(metrics)
    return 0.68 * predictive + 0.22 * usability_score + 0.10 * fair_use_bonus, predictive

def tune_threshold(y_val, scores_val, baseline_accuracy=None):
    best_t, best_s = 0.50, -1
    for t in np.linspace(0.20, 0.80, 31):
        m = metric_pack(y_val, scores_val, t)
        s = predictive_validation_score(m)
        # Guardrails: avoid thresholds that look absurd in viva.
        if m["precision"] < 0.12:
            s -= 0.15
        if m["accuracy"] < 0.55:
            s -= 0.12
        if baseline_accuracy is not None and m["accuracy"] < max(0.55, baseline_accuracy - 0.18):
            s -= 0.10
        if m["f1"] < 0.10:
            s -= 0.08
        if s > best_s:
            best_t, best_s = float(t), float(s)
    return best_t, best_s

def get_feature_names_from_pipeline(pipe):
    try:
        pre = pipe.named_steps["preprocess"]
        return list(pre.get_feature_names_out())
    except Exception:
        return []

def get_top_drivers(pipe, n=12):
    names = get_feature_names_from_pipeline(pipe)
    model = pipe.named_steps.get("model")
    values = None
    if hasattr(model, "feature_importances_"):
        values = model.feature_importances_
    elif hasattr(model, "coef_"):

```

```

values = np.abs(model.coef_).ravel()

if values is None or not names or len(values) != len(names):
    return []

candidates = []
for name, val in zip(names, values):
    clean = humanize_feature_name(name)
    if is_hr_business_driver(clean):
        candidates.append((clean, float(val)))
candidates = sorted(candidates, key=lambda x: abs(x[1]), reverse=True)
return candidates[:n]

def action_for_driver(driver):
    d = driver.lower()
    if "overtime" in d or "workload" in d:
        return "Run workload reviews, rebalance staffing where possible, and schedule stay interviews for overti
    if "work life" in d or "work-life" in d or "balance" in d:
        return "Run a pulse survey and manager coaching plan for teams with weaker work-life balance signals."
    if "satisfaction" in d or "environment" in d:
        return "Review engagement feedback, manager follow-through, recognition, and team climate for affected g
    if "remote" in d or "flex" in d:
        return "Review remote-work and flexibility patterns against engagement feedback and role requirements."
    if "promotion" in d or "growth" in d or "job level" in d or "level" in d:
        return "Review career progression, internal mobility, and promotion visibility for affected groups."
    if "distance" in d or "commute" in d or "home" in d:
        return "Explore commute flexibility, hybrid scheduling, or location-support options for affected employe
    if "income" in d or "salary" in d or "compensation" in d or "pay" in d or "rate" in d:
        return "Review compensation-band positioning and market competitiveness for affected groups."
    if "manager" in d:
        return "Coach managers on 1:1 quality, recognition, workload planning, and follow-through on employee co
    if "training" in d:
        return "Review training access, skill-building plans, and development conversations for affected groups.
    if "travel" in d:
        return "Review travel load, travel predictability, and support policies for roles with heavy travel dema
    if "tenure" in d or "years" in d:
        return "Add onboarding, 30/60/90-day check-ins, and early-tenure manager support for affected groups."
    return f"Review {driver} patterns with HRBP and managers, then validate with engagement and manager feedback

def generate_actions(drivers):
    actions = []
    seen = set()
    for driver, importance in drivers:
        topic = driver.split(":")[0].lower()
        if topic in seen:
            continue
        seen.add(topic)
        actions.append({
            "driver": driver,
            "action": action_for_driver(driver),
            "priority": "High" if len(actions) < 3 else "Medium",
            "why_it_matters": f"{driver} appears among the strongest validation drivers and may represent an act
        })
    if len(actions) >= 8:
        break
    return actions

def fair_use_check(df_original, usable_mask_index, sensitive_cols, scores, threshold, y_true=None):
    if not sensitive_cols:
        return {
            "status": "Not enough data to check",
            "sensitive_fields_excluded": "",
            "audited_groups": "",
            "largest_watchlist_gap": np.nan,
            "largest_error_gap": np.nan,
            "what_retainly_checked": "Retainly looked for sensitive attributes such as Gender, Age, Marital Stat
            "what_retainly_did": "No sensitive columns were available for group-level checking. Identity/leakage
            "what_retainly_verified": "Group-level fair-use concentration could not be tested because no sensiti
            "result": "Not enough data was available to run a group-level fair-use check.",
            "how_hr_should_use": "Use results only for supportive HR review, not punitive decisions.",
            "important_limit": "Do not use Retainly as the only basis for firing, demotion, salary cuts, discipl
            "fair_use_bonus": 0.60,
        }

    watchlist = (np.asarray(scores) >= threshold).astype(int)
    y_arr = np.asarray(y_true).astype(int) if y_true is not None else None

    checked = []
    max_gap = 0.0
    max_error_gap = 0.0

    # df_original is already filtered to usable rows before train/test split only in outer call for test subset.

```



```

for col in sensitive_cols:
    if col not in df_original.columns:
        continue
    s = df_original[col].reset_index(drop=True)
    if len(s) != len(watchlist):
        continue
    # Bucket numeric age-like sensitive columns so the audit is meaningful.
    if normalize_feature_key(col) == "age" or clean_name(col).lower() in {"employee age", "worker age"}:
        num = pd.to_numeric(s, errors="coerce")
        groups = pd.cut(num, bins=[-np.inf, 29, 44, np.inf], labels=["Under 30", "30 to 44", "45 plus"]).astype(str)
    else:
        groups = s.fillna("Unknown").astype(str)
    vc = groups.value_counts()
    valid_groups = [g for g, cnt in vc.items() if cnt >= 10]
    if len(valid_groups) < 2:
        continue

    rates = []
    error_rates = []
    for g in valid_groups:
        idx = groups.eq(g).values
        rates.append(float(watchlist[idx].mean()))
        if y_arr is not None:
            # simple group error rate difference for HR-readable safety check
            pred = watchlist
            error_rates.append(float((pred[idx] != y_arr[idx]).mean()))
    gap = max(rates) - min(rates) if rates else 0.0
    err_gap = max(error_rates) - min(error_rates) if len(error_rates) >= 2 else 0.0
    max_gap = max(max_gap, gap)
    max_error_gap = max(max_error_gap, err_gap)
    checked.append(col)

if not checked:
    return {
        "status": "Not enough data to check",
        "sensitive_fields_excluded": ", ".join(map(str, sensitive_cols)),
        "audited_groups": "",
        "largest_watchlist_gap": np.nan,
        "largest_error_gap": np.nan,
        "what_retainly_checked": "Retainly checked whether available sensitive attributes could unfairly inf",
        "what_retainly_did": "Sensitive fields were excluded from model training and scoring wherever presen",
        "what_retainly_verified": "Group-level fair-use concentration could not be tested because available",
        "result": "Not enough grouped data was available to complete the fair-use check.",
        "how_hr_should_use": "Use results only for supportive HR review, not punitive decisions.",
        "important_limit": "Do not use Retainly as the only basis for firing, demotion, salary cuts, discipl",
        "fair_use_bonus": 0.70,
    }

if max_gap <= 0.25 and max_error_gap <= 0.25:
    status = "Safe for supportive HR review"
    result = "No major unfair concentration was found in the selected fair-use checks. The recommendations c",
    bonus = 1.00
else:
    status = "Review before use"
    result = "Priority recommendations were unevenly concentrated across one or more available sensitive gro",
    bonus = 0.80

return {
    "status": status,
    "sensitive_fields_excluded": ", ".join(map(str, sensitive_cols)),
    "audited_groups": ", ".join(map(str, checked)),
    "largest_watchlist_gap": float(max_gap),
    "largest_error_gap": float(max_error_gap),
    "what_retainly_checked": "Retainly checked whether sensitive employee attributes could unfairly influenc",
    "what_retainly_did": "Sensitive fields such as Gender, Age, Marital Status, Race, Religion, and Disabili",
    "what_retainly_verified": "After scoring, Retainly checked whether priority recommendations were unfairl",
    "result": result,
    "how_hr_should_use": "Use Retainly for stay interviews, workload review, career-growth support, manager",
    "important_limit": "Do not use Retainly as the only basis for firing, demotion, salary cuts, disciplinar",
    "fair_use_bonus": bonus,
}

def run_one_dataset(path):
    df = pd.read_csv(path)
    target = detect_target_column(df)
    if target is None:
        raise ValueError(f"No attrition target found in {path.name}. Validation requires labeled datasets.")

    X, y, dropped, sensitive_cols, usable_df = prepare_xy(df, target)
    if y.nunique() < 2:
        raise ValueError(f"Target in {path.name} has only one class after normalization.")
    if len(y) < 100:

```

```

raise ValueError(f"{path.name} has too few usable rows for stable validation.")

# Keep original row ids aligned for fair-use test subset.
X = X.reset_index(drop=True)
y = y.reset_index(drop=True)
usable_df = usable_df.reset_index(drop=True)

if len(y) > 30000:
    sample_idx, _ = train_test_split(
        np.arange(len(y)), train_size=30000, stratify=y, random_state=RANDOM_STATE
    )
    X = X.iloc[sample_idx].reset_index(drop=True)
    y = y.iloc[sample_idx].reset_index(drop=True)
    usable_df = usable_df.iloc[sample_idx].reset_index(drop=True)

idx_all = np.arange(len(y))
trainval_idx, test_idx = train_test_split(
    idx_all, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)
train_idx, val_idx = train_test_split(
    trainval_idx, test_size=0.25, stratify=y.iloc[trainval_idx], random_state=RANDOM_STATE
)

X_train, X_val, X_test = X.iloc[train_idx], X.iloc[val_idx], X.iloc[test_idx]
y_train, y_val, y_test = y.iloc[train_idx], y.iloc[val_idx], y.iloc[test_idx]
test_original = usable_df.iloc[test_idx].reset_index(drop=True)

trace = [
    "Project Manager Agent: created a repeatable validation workflow and tracked each stage.",
    "Data Analyst Agent: detected the attrition target, removed ID/leakage/sensitive fields, and prepared fe
]

# Baseline
baseline = build_baseline_model(X_train)
baseline.fit(X_train, y_train)
base_scores = predict_scores(baseline, X_test)
base_metrics = metric_pack(y_test, base_scores, 0.50)
baseline_val_scores = predict_scores(baseline, X_val)
baseline_val_metrics = metric_pack(y_val, baseline_val_scores, 0.50)

# Retainly
candidates = build_retainly_candidates(X_train)
best = None
leaderboard = []
baseline_val_rank_floor = max(0.0, baseline_val_metrics.get("roc_auc", 0) - 0.05)
for name, model in candidates.items():
    model.fit(X_train, y_train)
    val_scores = predict_scores(model, X_val)
    t, s = tune_threshold(y_val, val_scores, baseline_accuracy=baseline_val_metrics["accuracy"])
    val_m = metric_pack(y_val, val_scores, t)

    # Strong anti-garbage guard: do not pick a model with much worse ranking than the baseline.
    if val_m.get("roc_auc", 0) < baseline_val_rank_floor:
        s -= 0.20
    if val_m.get("pr_auc", 0) < max(0.0, baseline_val_metrics.get("pr_auc", 0) - 0.08):
        s -= 0.08

    leaderboard.append({"model": name, "threshold": t, "score": s, **val_m})
    if best is None or s > best["score"]:
        best = {"name": name, "model": model, "threshold": t, "score": s, "val_metrics": val_m}

trace.append(f"ML Engineer Agent: compared {len(candidates)} candidate models and selected {best['name']} at

retain_scores = predict_scores(best["model"], X_test)
retain_metrics = metric_pack(y_test, retain_scores, best["threshold"])

fair = fair_use_check(test_original, None, sensitive_cols, retain_scores, best["threshold"], y_true=y_test.v
trace.append(f"Insights Agent: generated drivers, HR actions, charts, and Fair Use Check status: {fair['stat

base_final, base_pred_score = final_project_score(base_metrics, usability_score=0.25, fair_use_bonus=0.25)
retain_final, retain_pred_score = final_project_score(retain_metrics, usability_score=1.00, fair_use_bonus=f

drivers = get_top_drivers(best["model"], n=12)
actions = generate_actions(drivers)

dataset_name = path.stem
if "synthetic" in dataset_name.lower():
    dataset_label = f"{dataset_name} (synthetic stress-test)"
else:
    dataset_label = dataset_name

attr_rate = float(y.mean())

```

```

rows = []
for approach, metrics, selected_model, threshold, pred_score, usability, final_score in [
    ("Baseline", base_metrics, "LogisticRegression", 0.50, base_pred_score, 0.25, base_final),
    ("Retainly", retain_metrics, best["name"], best["threshold"], retain_pred_score, 1.00, retain_final),
]:
    row = {
        "dataset": dataset_name,
        "dataset_label": dataset_label,
        "rows": int(len(df)),
        "usable_rows": int(len(y)),
        "columns": int(len(df.columns)),
        "target_column": target,
        "attrition_rate": attr_rate,
        "approach": approach,
        "selected_model": selected_model,
        "selected_threshold": threshold,
        "predictive_score": pred_score,
        "usability_score": usability,
        "fair_use_score": 0.25 if approach == "Baseline" else fair["fair_use_bonus"],
        "fair_use_status": "Not part of baseline workflow" if approach == "Baseline" else fair["status"],
        "final_project_score": final_score,
    }
    row.update(metrics)
    rows.append(row)

fair_row = {
    "dataset": dataset_name,
    **{k: v for k, v in fair.items() if k != "fair_use_bonus"}
}

return {
    "dataset": dataset_name,
    "results": rows,
    "drivers": [{"dataset": dataset_name, "driver": d, "importance": imp} for d, imp in drivers],
    "actions": [{"dataset": dataset_name, **a} for a in actions],
    "trace": [{"dataset": dataset_name, "message": m} for m in trace],
    "leaderboard": [{"dataset": dataset_name, **r} for r in leaderboard],
    "dropped_columns": [{"dataset": dataset_name, "dropped_column": c} for c in dropped],
    "fair_use": [fair_row],
}

```

2. Run validation comparison

The notebook runs on every labeled CSV uploaded into `research_datasets/`.

Unlabeled or invalid files are skipped with a clear reason.

```

csv_files = sorted(DATA_DIR.glob("*.csv"))
if not csv_files:
    raise RuntimeError("No CSV files found. Upload at least one labeled attrition CSV.")

all_results, all_drivers, all_actions, all_trace, all_leaderboard, all_dropped, all_fair, skipped = [], [], [],
for path in csv_files:
    print("\nRunning:", path.name)
    try:
        comp = run_one_dataset(path)
        all_results.extend(comp["results"])
        all_drivers.extend(comp["drivers"])
        all_actions.extend(comp["actions"])
        all_trace.extend(comp["trace"])
        all_leaderboard.extend(comp["leaderboard"])
        all_dropped.extend(comp["dropped_columns"])
        all_fair.extend(comp["fair_use"])
        print(" OK:", comp["dataset"])
    except Exception as e:
        skipped.append({"file": path.name, "reason": str(e)})
        print(" SKIPPED:", e)

if not all_results:
    raise RuntimeError("No labeled datasets could be validated. Check target column names and labels.")

results = pd.DataFrame(all_results)
drivers_df = pd.DataFrame(all_drivers)
actions_df = pd.DataFrame(all_actions)
trace_df = pd.DataFrame(all_trace)
leaderboard_df = pd.DataFrame(all_leaderboard)
dropped_df = pd.DataFrame(all_dropped)

```

```

fair_use_df = pd.DataFrame(all_fair)
skipped_df = pd.DataFrame(skipped)

results.to_csv(OUT_DIR / "dataset_comparison_results.csv", index=False)
drivers_df.to_csv(OUT_DIR / "top_drivers_summary.csv", index=False)
actions_df.to_csv(OUT_DIR / "hr_actions_summary.csv", index=False)
trace_df.to_csv(OUT_DIR / "agent_trace_summary.csv", index=False)
leaderboard_df.to_csv(OUT_DIR / "retainly_model_leaderboard.csv", index=False)
dropped_df.to_csv(OUT_DIR / "dropped_columns_summary.csv", index=False)
fair_use_df.to_csv(OUT_DIR / "fair_use_summary.csv", index=False)
skipped_df.to_csv(OUT_DIR / "skipped_datasets.csv", index=False)

display(results.sort_values(["dataset", "approach"]))
if not skipped_df.empty:
    print("Skipped datasets")
    display(skipped_df)

```

Running: employee_attrition_datasettrain.csv
OK: employee_attrition_datasettrain

Running: ibm_hr_attrition.csv
OK: ibm_hr_attrition

Running: synthetic_employee_attrition_74498train.csv
OK: synthetic_employee_attrition_74498train

	dataset	dataset_label	rows	usable_rows	columns	target_column	attr
0	employee_attrition_datasettrain	employee_attrition_datasettrain	1000	1000	26	Attrition	
1	employee_attrition_datasettrain	employee_attrition_datasettrain	1000	1000	26	Attrition	
2	ibm_hr_attrition	ibm_hr_attrition	1470	1470	35	Attrition	
3	ibm_hr_attrition	ibm_hr_attrition	1470	1470	35	Attrition	
4	synthetic_employee_attrition_74498train	synthetic_employee_attrition_74498train (synth...	59598	30000	24	Attrition	
5	synthetic_employee_attrition_74498train	synthetic_employee_attrition_74498train (synth...	59598	30000	24	Attrition	

6 rows x 25 columns

3. Average result summary

Read this honestly:

- Baseline may win accuracy/precision.
- Retainly should be judged as an attrition intelligence workflow.
- Recall, F1, PR-AUC, top-k prioritization, Fair Use Check, and HR actions matter for this project.

```

metric_cols = [
    "accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc",
    "recall_at_top_10_percent", "recall_at_top_20_percent",
    "lift_at_top_10_percent", "lift_at_top_20_percent",
    "predictive_score", "usability_score", "fair_use_score", "final_project_score"
]
avg = results.groupby("approach")[metric_cols].mean(numeric_only=True).reset_index()
display(avg)

summary = {
    "datasets_run": sorted(results["dataset"].unique().tolist()),
    "skipped_datasets": skipped,
    "average_by_approach": avg.to_dict("records"),
}

if "Baseline" in avg["approach"].values and "Retainly" in avg["approach"].values:
    b = avg[avg["approach"].eq("Baseline")].drop(columns=["approach"]).to_dict("records")[0]
    r = avg[avg["approach"].eq("Retainly")].drop(columns=["approach"]).to_dict("records")[0]
    summary["average_baseline_metrics"] = b
    summary["average_retainly_metrics"] = r
    summary["average_lift"] = {k: r.get(k, 0) - b.get(k, 0) for k in r.keys()}
    summary["best_overall_approach"] = "Retainly" if r.get("final_project_score", 0) >= b.get("final_project_sco

summary["conclusion"] = (
    "Retainly is evaluated as a multi-agent attrition decision-support workflow. "
    "Baseline models may be stronger on accuracy or precision on some datasets. "
    "Retainly is designed for HR intervention, where recall, F1, PR-AUC, probability-ranked top-k prioritization
    "explainable drivers, HR action planning, and a Fair Use Check matter. "
    "The final project score summarizes predictive performance and decision-support value."

```

```
)  
  
with open(OUT_DIR / "dataset_comparison_summary.json", "w") as f:  
    json.dump(summary, f, indent=2)  
  
summary
```

Next steps:

[Generate code with avg](#)[New interactive sheet](#)

	approach	accuracy	precision	recall	f1	roc_auc	pr_auc	recall_at_top_10_percent	recall_at_top_20_p
0	Baseline	0.796939	0.632369	0.362310	0.407560	0.677024	0.512735	0.229732	0

4. Generate validation charts

```
plt.rcParams.update({"figure.dpi": 150, "font.size": 10})

def add_labels(ax, bars, fmt="{:.2f}"):
    for b in bars:
        h = b.get_height()
        ax.annotate(fmt.format(h), (b.get_x() + b.get_width()/2, h),
                    ha="center", va="bottom", fontsize=8, xytext=(0, 3), textcoords="offset points")

# Core metric chart
core_metrics = ["accuracy", "precision", "recall", "f1", "roc_auc", "pr_auc", "final_project_score"]
core_avg = results.groupby("approach")[core_metrics].mean(numeric_only=True).reindex(["Baseline", "Retainly"])
x = np.arange(len(core_metrics))
w = 0.36
fig, ax = plt.subplots(figsize=(9, 4.6))
b1 = ax.bar(x - w/2, core_avg.loc["Baseline"], w, label="Baseline")
b2 = ax.bar(x + w/2, core_avg.loc["Retainly"], w, label="Retainly")
ax.set_ylim(0, 1.08)
ax.set_ylabel("Average score")
ax.set_title("Core validation metrics")
ax.set_xticks(x)
ax.set_xticklabels(["Accuracy", "Precision", "Recall", "F1", "ROC-AUC", "PR-AUC", "Final score"], rotation=25, ha="right")
ax.legend()
add_labels(ax, b1)
add_labels(ax, b2)
fig.tight_layout()
fig.savefig(OUT_DIR / "dataset_comparison_core_metrics.png", bbox_inches="tight")
plt.show()

# Top-k chart
topk_metrics = ["recall_at_top_10_percent", "recall_at_top_20_percent", "lift_at_top_10_percent", "lift_at_top_20_percent"]
topk_avg = results.groupby("approach")[topk_metrics].mean(numeric_only=True).reindex(["Baseline", "Retainly"])
fig, axes = plt.subplots(1, 2, figsize=(9, 4.2))
panels = [
    (axes[0], topk_metrics[2:], ["Recall@Top10", "Recall@Top20", "Probability-ranked recall"]),
    (axes[1], topk_metrics[0:2], ["Lift@Top10", "Lift@Top20", "Probability-ranked lift"]),
]
for ax, cols, labels, title in panels:
    xx = np.arange(len(cols))
    b1 = ax.bar(xx - w/2, topk_avg.loc["Baseline", cols], w, label="Baseline")
    b2 = ax.bar(xx + w/2, topk_avg.loc["Retainly", cols], w, label="Retainly")
    ax.set_title(title)
    ax.set_xticks(xx)
    ax.set_xticklabels(labels, rotation=15, ha="right")
    ax.set_ylabel("Average score")
    ymax = max(float(topk_avg[cols].max().max()) * 1.25, 1.0)
    ax.set_ylim(0, ymax)
    ax.legend()
    add_labels(ax, b1)
    add_labels(ax, b2)
fig.suptitle("Top-k prioritization metrics")
fig.tight_layout()
fig.savefig(OUT_DIR / "dataset_comparison_topk_metrics.png", bbox_inches="tight")
plt.show()

# Final score chart
final_avg = results.groupby("approach")["final_project_score"].mean(numeric_only=True).reindex(["Baseline", "Retainly"])
fig, ax = plt.subplots(figsize=(6.5, 4.2))
bars = ax.bar(final_avg.index, final_avg.values)
ax.set_ylim(0, 1.08)
ax.set_ylabel("Average score")
ax.set_title("Final decision-support score")
add_labels(ax, bars)
fig.tight_layout()
fig.savefig(OUT_DIR / "dataset_comparison_final_score.png", bbox_inches="tight")
plt.show()

# Fair use chart
if not fair_use_df.empty:
    status_counts = fair_use_df["status"].value_counts()
    fig, ax = plt.subplots(figsize=(7, 3.8))
    bars = ax.bar(status_counts.index, status_counts.values)
    ax.set_ylabel("Dataset count")
    ax.set_title("Fair Use Check summary")
    add_labels(ax, bars, fmt="{:.0f}")
    fig.tight_layout()
    fig.savefig(OUT_DIR / "fair_use_check_summary.png", bbox_inches="tight")
    plt.show()
```

```
print("Charts saved in:", OUT_DIR.resolve())
```


